Volume 06 Specifications - Knowledge Capability Workflow Specifications

Version v29 draft

README.md

Knowledge Capability Workflow Specifications

Version: v29 draft

Status: Draft

Document ID: MAL-V06-V29-README

Purpose

This release folder contains the official v29 documentation set for Knowledge Capability Workflow Specifications.

Scope

- Covers Knowledge System.
- Covers Capability System.
- Covers Workflow System.

Acceptance

- All documents contain implementation-level guidance.
- The release PDF and ZIP are generated and verified.
- MANIFEST and RELEASE_NOTES are updated for v29.

Knowledge Capability Workflow Specifications Index

spec-06-29-01-knowledge-system.md

Knowledge System Specification

Title: Knowledge System Specification

Version: v29 draft

Status: Draft

Specification ID: MAL-V06-KNOWLEDGE-SYSTEM-SPEC-01

Related Blueprint Documents

- Volume 03 Master Blueprint
- Volume 04 Canonical Dictionary
- Volume 05 Engineering Standards
- Prior Volume 06 Specifications

Purpose

Define implementation-level requirements for Knowledge System in Mariam AI Enterprise OS so engineering teams can build, test, operate, and govern it consistently.

Scope

- Covers system responsibilities, runtime behavior, interfaces, events, storage, security, observability, and acceptance evidence.
- Includes interactions with Enterprise Core, AI Society, Runtime Ecosystem, Knowledge System, Capability System, Governance, and Infrastructure.
- Excludes application-specific code, vendor credentials, and temporary implementation shortcuts.

Responsibilities

- Maintain authoritative state and lifecycle rules for Knowledge System.
- Validate inputs, permissions, dependencies, and policy constraints before execution.
- Emit auditable events for lifecycle transitions, failures, recovery, and acceptance.
- Provide deterministic behavior that can be tested and traced across releases.

Functional Requirements

- SHALL expose versioned public interfaces for create, read, update, execute, validate, and audit operations.
- SHALL validate all inbound requests against schema, identity, policy, and dependency rules.
- SHALL support idempotency keys for commands that may be retried.
- SHALL publish events for accepted, rejected, started, completed, failed, recovered, and archived states.
- SHALL produce evidence bundles for review, compliance, and release acceptance.

Non-Functional Requirements

- MUST preserve tenant isolation, least privilege, and traceable authorization.
- MUST support observable latency, throughput, failure rate, queue depth, and recovery metrics.

- MUST recover safely after process restart, duplicate event delivery, or partial storage failure.
- SHOULD degrade gracefully when optional providers, plugins, connectors, or models are unavailable.

Inputs

- User, agent, workflow, runtime, provider, plugin, connector, and governance requests.
- Configuration, policies, feature flags, schemas, secrets references, and dependency metadata.
- Runtime events, task graph state, knowledge records, capability scores, and audit context.

Outputs

- Versioned records, execution results, validation reports, events, metrics, traces, logs, and acceptance evidence.
- Human-readable status summaries for dashboards, operations, release notes, and incident review.

Public Interfaces

- create_knowledge_system(request)
- get_knowledge_system_state(id)
- validate_knowledge_system(payload)
- execute_knowledge_system(command)
- audit_knowledge_system(scope)

Internal Interfaces

- Event Bus for durable event publication and subscription.
- Service Container for dependency injection and lifecycle management.
- Runtime Manager for execution dispatch and cancellation.
- Storage layer for records, snapshots, indexes, and evidence.
- Governance service for approval, policy, risk, and compliance checks.

Events

- knowledge-system.created
- knowledge-system.validated
- knowledge-system.started
- knowledge-system.completed
- knowledge-system.failed
- knowledge-system.recovered
- knowledge-system.accepted

Storage Requirements

- Store canonical records with identifiers, owner, version, state, timestamps, policy version, and trace ID.
- Store immutable event history and mutable read models separately where practical.
- Store evidence, validation reports, and recovery metadata with retention rules.

Security Requirements

- Enforce authentication, authorization, tenant isolation, and field-level redaction.
- Protect secrets by reference only; never persist raw credentials in logs, events, Markdown, or exports.
- Require approval for privileged, destructive, externally visible, financial, or compliance-sensitive actions.

Observability Requirements

- Emit structured logs with correlation IDs and decision reasons.
- Emit metrics for request count, success rate, failure rate, latency, retries, recovery success, and acceptance failures.
- Provide traces across public interface, policy validation, runtime execution, storage, events, and callbacks.

Failure Modes

- Invalid input, missing dependency, expired permission, unavailable runtime, conflicting state, duplicate command, or failed downstream provider.
- Incomplete event delivery, partial persistence, timeout, stalled execution, or acceptance failure.

Recovery Rules

- Retry idempotent commands with bounded backoff.
- Rebuild read models from event history when projections drift.
- Escalate to governance when automatic recovery changes risk, scope, budget, deadline, or external side effects.
- Preserve failed attempts and recovery decisions as audit evidence.

Test Requirements

- Unit tests for schemas, state transitions, permission checks, and event payloads.
- Integration tests for public interfaces, storage, event bus, runtime dependencies, and governance approvals.
- Failure tests for retries, duplicate events, unavailable dependencies, and recovery paths.
- Acceptance tests proving traceability from requirement to evidence.

Acceptance Criteria

- The specification is complete enough to create implementation tickets without hidden architectural decisions.
- Interfaces, events, storage, security, observability, failures, recovery, and tests are documented.
- The document traces to Blueprint, Standards, and release artifacts.

Implementation Notes

- Prefer small versioned interfaces and append-only event history.
- Keep provider-specific details behind adapters.
- Treat auditability and recovery as first-class design constraints.
- Validate schemas before work reaches runtime execution.

Traceability

Source	Relationship
MAL-V06-KNOWLEDGE-SYSTEM-SPEC-01	Defines v29 implementation requirements for Knowledge System.

Source	Relationship
Volume 03	Blueprint source.
Volume 05	Engineering standards source.
RELEASE_NOTES_v29_draft.md	Release evidence.

Version History

Version	Date	Status	Notes
v29 draft	2026-06-29	Draft	Initial specification for Knowledge System.

spec-06-29-02-capability-system.md

Capability System Specification

Title: Capability System Specification

Version: v29 draft

Status: Draft

Specification ID: MAL-V06-CAPABILITY-SYSTEM-SPEC-02

Related Blueprint Documents

- Volume 03 Master Blueprint
- Volume 04 Canonical Dictionary
- Volume 05 Engineering Standards
- Prior Volume 06 Specifications

Purpose

Define implementation-level requirements for Capability System in Mariam AI Enterprise OS so engineering teams can build, test, operate, and govern it consistently.

Scope

- Covers system responsibilities, runtime behavior, interfaces, events, storage, security, observability, and acceptance evidence.
- Includes interactions with Enterprise Core, AI Society, Runtime Ecosystem, Knowledge System, Capability System, Governance, and Infrastructure.
- Excludes application-specific code, vendor credentials, and temporary implementation shortcuts.

Responsibilities

- Maintain authoritative state and lifecycle rules for Capability System.
- Validate inputs, permissions, dependencies, and policy constraints before execution.
- Emit auditable events for lifecycle transitions, failures, recovery, and acceptance.
- Provide deterministic behavior that can be tested and traced across releases.

Functional Requirements

- SHALL expose versioned public interfaces for create, read, update, execute, validate, and audit operations.
- SHALL validate all inbound requests against schema, identity, policy, and dependency rules.
- SHALL support idempotency keys for commands that may be retried.
- SHALL publish events for accepted, rejected, started, completed, failed, recovered, and archived states.
- SHALL produce evidence bundles for review, compliance, and release acceptance.

Non-Functional Requirements

- MUST preserve tenant isolation, least privilege, and traceable authorization.
- MUST support observable latency, throughput, failure rate, queue depth, and recovery metrics.

- MUST recover safely after process restart, duplicate event delivery, or partial storage failure.
- SHOULD degrade gracefully when optional providers, plugins, connectors, or models are unavailable.

Inputs

- User, agent, workflow, runtime, provider, plugin, connector, and governance requests.
- Configuration, policies, feature flags, schemas, secrets references, and dependency metadata.
- Runtime events, task graph state, knowledge records, capability scores, and audit context.

Outputs

- Versioned records, execution results, validation reports, events, metrics, traces, logs, and acceptance evidence.
- Human-readable status summaries for dashboards, operations, release notes, and incident review.

Public Interfaces

- create_capability_system(request)
- get_capability_system_state(id)
- validate_capability_system(payload)
- execute_capability_system(command)
- audit_capability_system(scope)

Internal Interfaces

- Event Bus for durable event publication and subscription.
- Service Container for dependency injection and lifecycle management.
- Runtime Manager for execution dispatch and cancellation.
- Storage layer for records, snapshots, indexes, and evidence.
- Governance service for approval, policy, risk, and compliance checks.

Events

- capability-system.created
- capability-system.validated
- capability-system.started
- capability-system.completed
- capability-system.failed
- capability-system.recovered
- capability-system.accepted

Storage Requirements

- Store canonical records with identifiers, owner, version, state, timestamps, policy version, and trace ID.
- Store immutable event history and mutable read models separately where practical.
- Store evidence, validation reports, and recovery metadata with retention rules.

Security Requirements

- Enforce authentication, authorization, tenant isolation, and field-level redaction.
- Protect secrets by reference only; never persist raw credentials in logs, events, Markdown, or exports.
- Require approval for privileged, destructive, externally visible, financial, or compliance-sensitive actions.

Observability Requirements

- Emit structured logs with correlation IDs and decision reasons.
- Emit metrics for request count, success rate, failure rate, latency, retries, recovery success, and acceptance failures.
- Provide traces across public interface, policy validation, runtime execution, storage, events, and callbacks.

Failure Modes

- Invalid input, missing dependency, expired permission, unavailable runtime, conflicting state, duplicate command, or failed downstream provider.
- Incomplete event delivery, partial persistence, timeout, stalled execution, or acceptance failure.

Recovery Rules

- Retry idempotent commands with bounded backoff.
- Rebuild read models from event history when projections drift.
- Escalate to governance when automatic recovery changes risk, scope, budget, deadline, or external side effects.
- Preserve failed attempts and recovery decisions as audit evidence.

Test Requirements

- Unit tests for schemas, state transitions, permission checks, and event payloads.
- Integration tests for public interfaces, storage, event bus, runtime dependencies, and governance approvals.
- Failure tests for retries, duplicate events, unavailable dependencies, and recovery paths.
- Acceptance tests proving traceability from requirement to evidence.

Acceptance Criteria

- The specification is complete enough to create implementation tickets without hidden architectural decisions.
- Interfaces, events, storage, security, observability, failures, recovery, and tests are documented.
- The document traces to Blueprint, Standards, and release artifacts.

Implementation Notes

- Prefer small versioned interfaces and append-only event history.
- Keep provider-specific details behind adapters.
- Treat auditability and recovery as first-class design constraints.
- Validate schemas before work reaches runtime execution.

Traceability

Source	Relationship
MAL-V06-CAPABILITY-SYSTEM-SPEC-02	Defines v29 implementation requirements for Capability System.

Source	Relationship
Volume 03	Blueprint source.
Volume 05	Engineering standards source.
RELEASE_NOTES_v29_draft.md	Release evidence.

Version History

Version	Date	Status	Notes
v29 draft	2026-06-29	Draft	Initial specification for Capability System.

spec-06-29-03-workflow-system.md

Workflow System Specification

Title: Workflow System Specification

Version: v29 draft

Status: Draft

Specification ID: MAL-V06-WORKFLOW-SYSTEM-SPEC-03

Related Blueprint Documents

- Volume 03 Master Blueprint
- Volume 04 Canonical Dictionary
- Volume 05 Engineering Standards
- Prior Volume 06 Specifications

Purpose

Define implementation-level requirements for Workflow System in Mariam AI Enterprise OS so engineering teams can build, test, operate, and govern it consistently.

Scope

- Covers system responsibilities, runtime behavior, interfaces, events, storage, security, observability, and acceptance evidence.
- Includes interactions with Enterprise Core, AI Society, Runtime Ecosystem, Knowledge System, Capability System, Governance, and Infrastructure.
- Excludes application-specific code, vendor credentials, and temporary implementation shortcuts.

Responsibilities

- Maintain authoritative state and lifecycle rules for Workflow System.
- Validate inputs, permissions, dependencies, and policy constraints before execution.
- Emit auditable events for lifecycle transitions, failures, recovery, and acceptance.
- Provide deterministic behavior that can be tested and traced across releases.

Functional Requirements

- SHALL expose versioned public interfaces for create, read, update, execute, validate, and audit operations.
- SHALL validate all inbound requests against schema, identity, policy, and dependency rules.
- SHALL support idempotency keys for commands that may be retried.
- SHALL publish events for accepted, rejected, started, completed, failed, recovered, and archived states.
- SHALL produce evidence bundles for review, compliance, and release acceptance.

Non-Functional Requirements

- MUST preserve tenant isolation, least privilege, and traceable authorization.
- MUST support observable latency, throughput, failure rate, queue depth, and recovery metrics.

- MUST recover safely after process restart, duplicate event delivery, or partial storage failure.
- SHOULD degrade gracefully when optional providers, plugins, connectors, or models are unavailable.

Inputs

- User, agent, workflow, runtime, provider, plugin, connector, and governance requests.
- Configuration, policies, feature flags, schemas, secrets references, and dependency metadata.
- Runtime events, task graph state, knowledge records, capability scores, and audit context.

Outputs

- Versioned records, execution results, validation reports, events, metrics, traces, logs, and acceptance evidence.
- Human-readable status summaries for dashboards, operations, release notes, and incident review.

Public Interfaces

- create_workflow_system(request)
- get_workflow_system_state(id)
- validate_workflow_system(payload)
- execute_workflow_system(command)
- audit_workflow_system(scope)

Internal Interfaces

- Event Bus for durable event publication and subscription.
- Service Container for dependency injection and lifecycle management.
- Runtime Manager for execution dispatch and cancellation.
- Storage layer for records, snapshots, indexes, and evidence.
- Governance service for approval, policy, risk, and compliance checks.

Events

- workflow-system.created
- workflow-system.validated
- workflow-system.started
- workflow-system.completed
- workflow-system.failed
- workflow-system.recovered
- workflow-system.accepted

Storage Requirements

- Store canonical records with identifiers, owner, version, state, timestamps, policy version, and trace ID.
- Store immutable event history and mutable read models separately where practical.
- Store evidence, validation reports, and recovery metadata with retention rules.

Security Requirements

- Enforce authentication, authorization, tenant isolation, and field-level redaction.
- Protect secrets by reference only; never persist raw credentials in logs, events, Markdown, or exports.
- Require approval for privileged, destructive, externally visible, financial, or compliance-sensitive actions.

Observability Requirements

- Emit structured logs with correlation IDs and decision reasons.
- Emit metrics for request count, success rate, failure rate, latency, retries, recovery success, and acceptance failures.
- Provide traces across public interface, policy validation, runtime execution, storage, events, and callbacks.

Failure Modes

- Invalid input, missing dependency, expired permission, unavailable runtime, conflicting state, duplicate command, or failed downstream provider.
- Incomplete event delivery, partial persistence, timeout, stalled execution, or acceptance failure.

Recovery Rules

- Retry idempotent commands with bounded backoff.
- Rebuild read models from event history when projections drift.
- Escalate to governance when automatic recovery changes risk, scope, budget, deadline, or external side effects.
- Preserve failed attempts and recovery decisions as audit evidence.

Test Requirements

- Unit tests for schemas, state transitions, permission checks, and event payloads.
- Integration tests for public interfaces, storage, event bus, runtime dependencies, and governance approvals.
- Failure tests for retries, duplicate events, unavailable dependencies, and recovery paths.
- Acceptance tests proving traceability from requirement to evidence.

Acceptance Criteria

- The specification is complete enough to create implementation tickets without hidden architectural decisions.
- Interfaces, events, storage, security, observability, failures, recovery, and tests are documented.
- The document traces to Blueprint, Standards, and release artifacts.

Implementation Notes

- Prefer small versioned interfaces and append-only event history.
- Keep provider-specific details behind adapters.
- Treat auditability and recovery as first-class design constraints.
- Validate schemas before work reaches runtime execution.

Traceability

Source	Relationship
MAL-V06-WORKFLOW-SYSTEM-SPEC-03	Defines v29 implementation requirements for Workflow System.

Source	Relationship
Volume 03	Blueprint source.
Volume 05	Engineering standards source.
RELEASE_NOTES_v29_draft.md	Release evidence.

Version History

Version	Date	Status	Notes
v29 draft	2026-06-29	Draft	Initial specification for Workflow System.